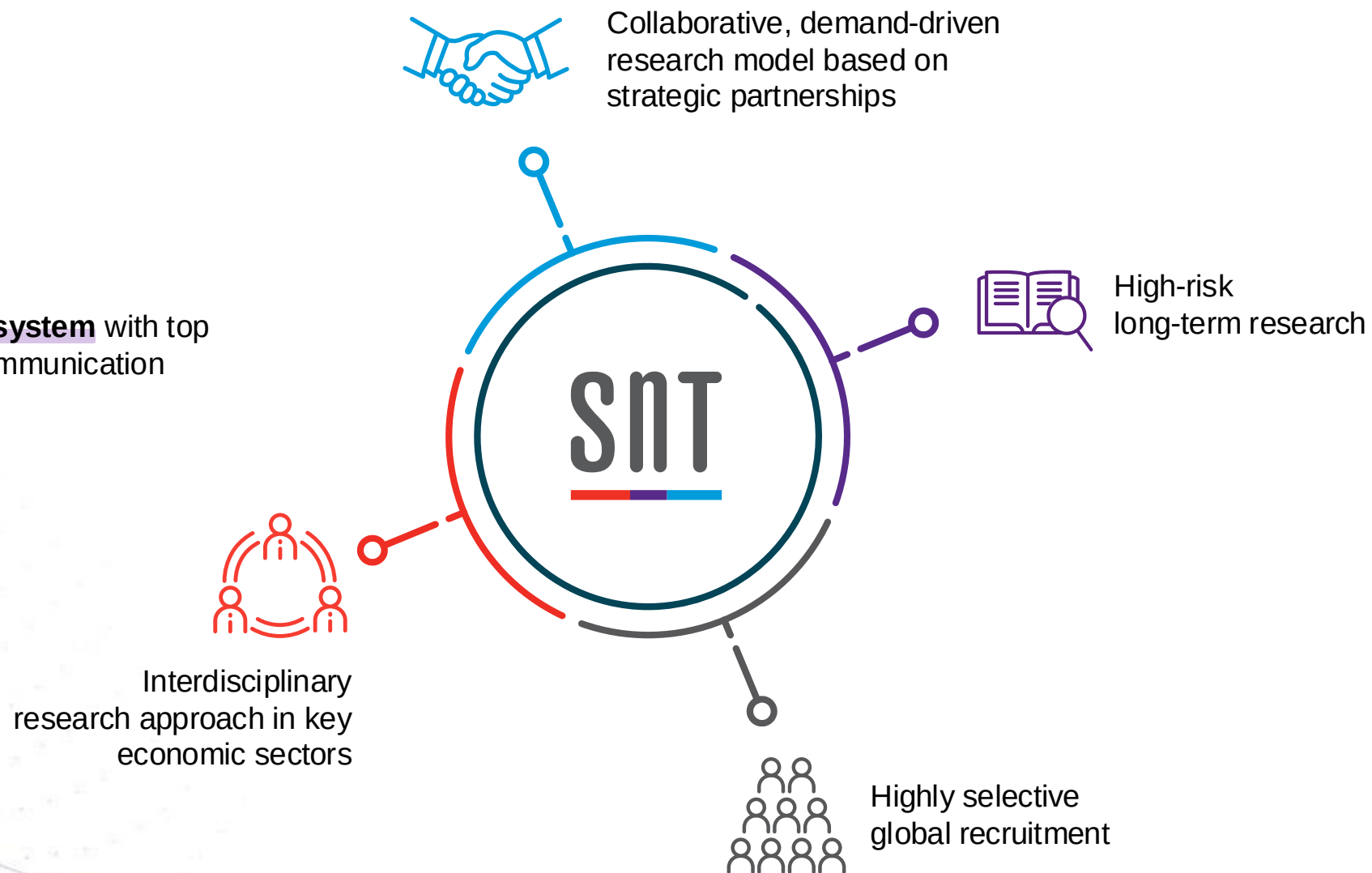


SNT Software V&V in the age of GenAI

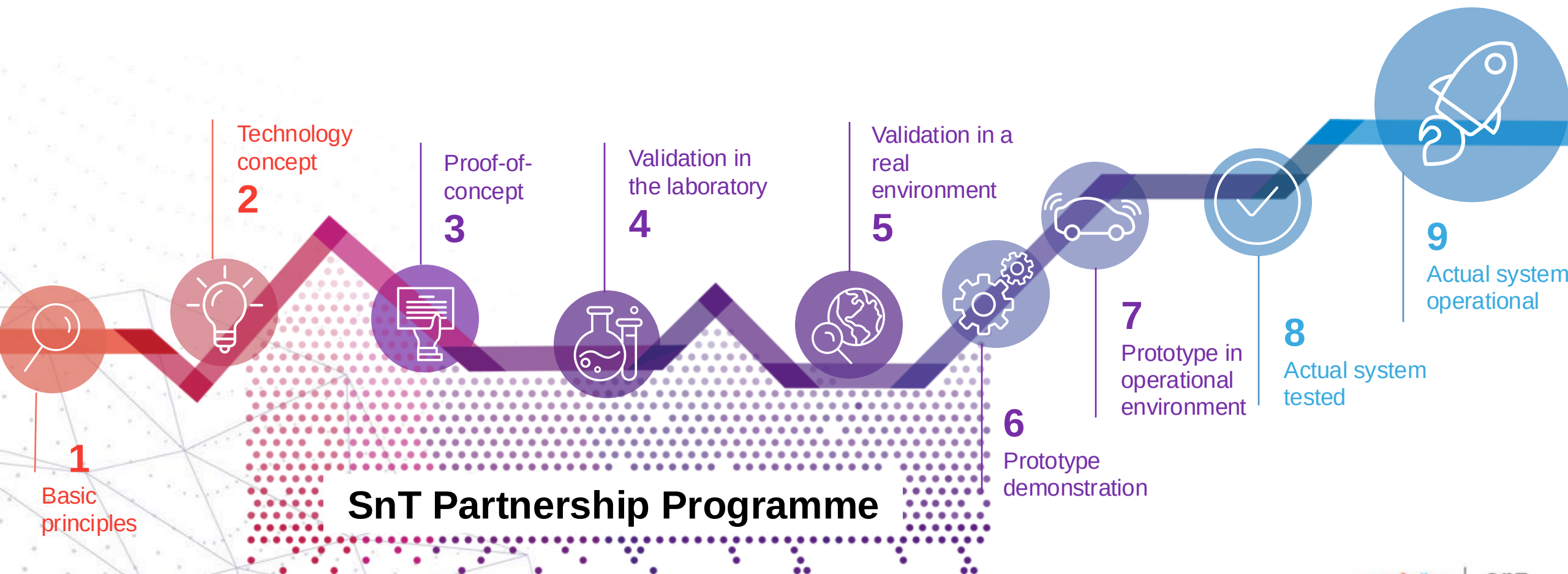
Prof. Domenico Bianculli
Prof. Fabrizio Pastore

We fuel the national innovation system with top level research in Information & Communication Technologies.



SnT Partnership Programme

We are guided by the principle that excellent scientific research can address the most pressing challenges society faces, and support industry in **developing solutions**. We have a partnership model that enables **truly collaborative** exchanges, allowing us to have access to relevant challenges, real-world data, and systems to test our research results.



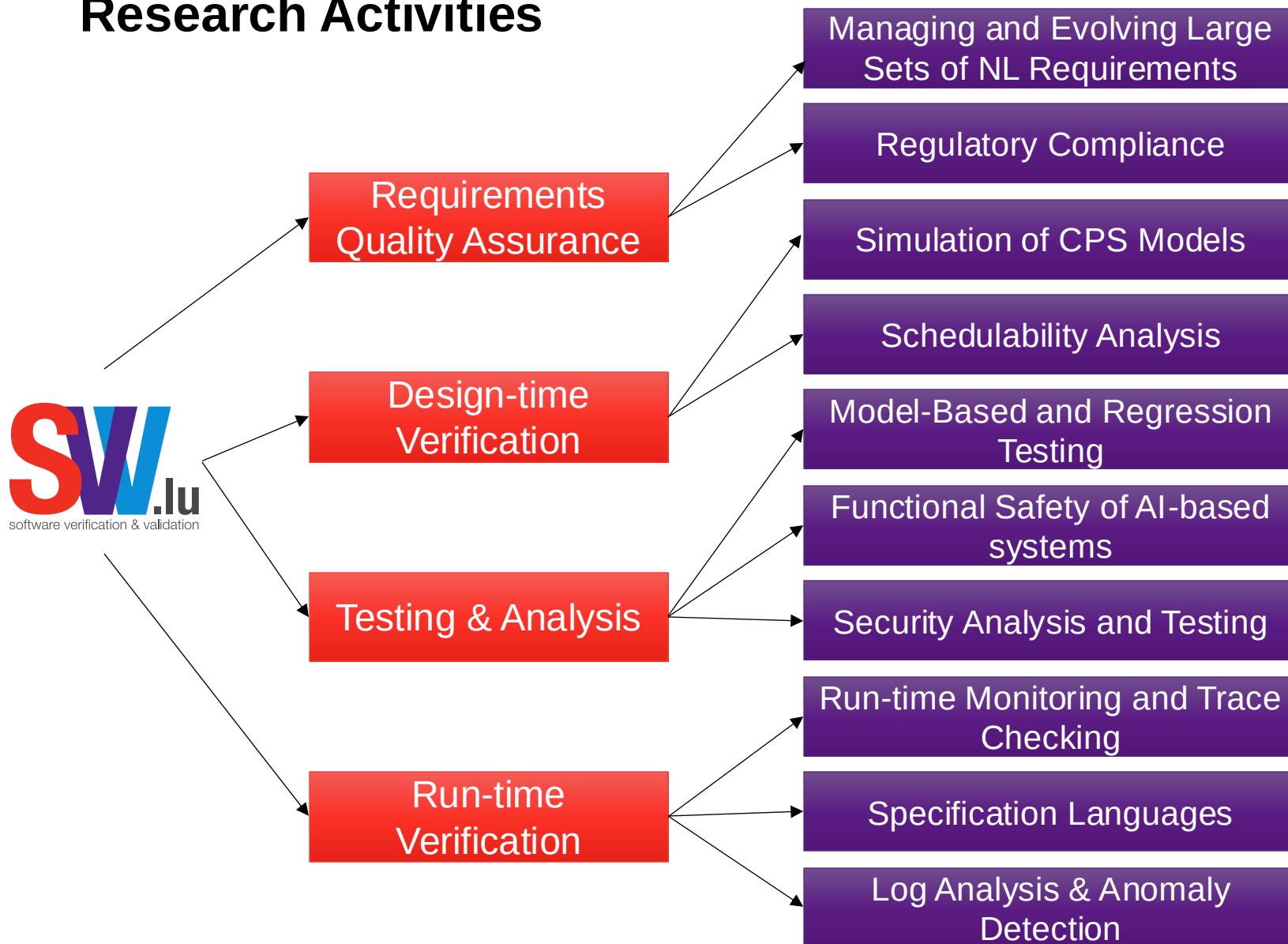
Software Verification & Validation (SVV) Research Group

- Head of the group: Prof. Domenico Bianculli
- Deputy Head of the group: Prof. Fabrizio Pastore
- 14 members
 - 2 faculty members
 - 2 research scientists
 - 6 research associates
 - 4 PhD candidates

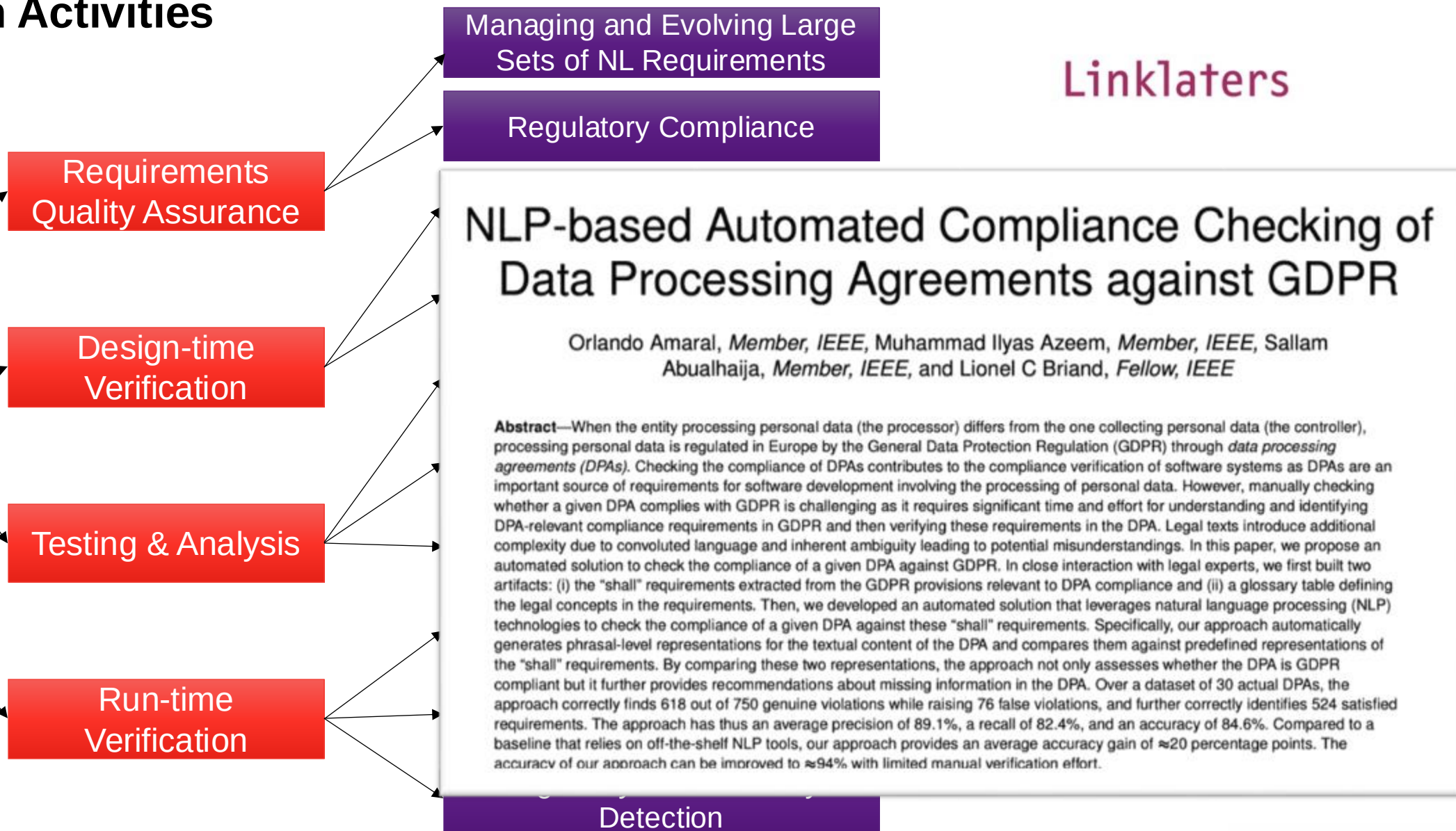


www.svv.lu

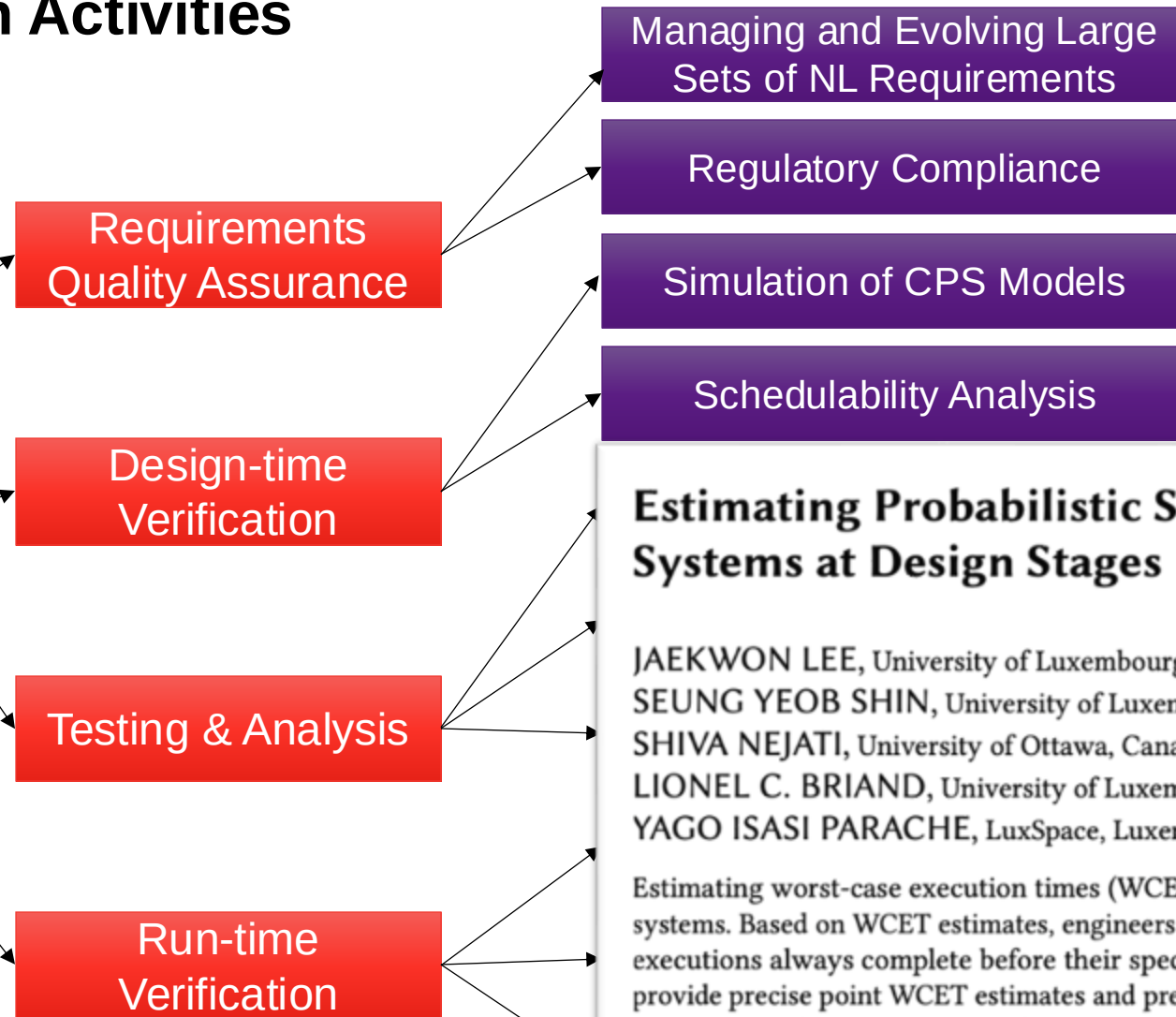
Research Activities



Research Activities



Research Activities



Estimating Probabilistic Safe WCET Ranges of Real-Time Systems at Design Stages

JAEKWON LEE, University of Luxembourg, Luxembourg and University of Ottawa, Canada
 SEUNG YEOB SHIN, University of Luxembourg, Luxembourg
 SHIVA NEJATI, University of Ottawa, Canada and University of Luxembourg, Luxembourg
 LIONEL C. BRIAND, University of Luxembourg, Luxembourg and University of Ottawa, Canada
 YAGO ISASI PARACHE, LuxSpace, Luxembourg

Estimating worst-case execution times (WCET) is an important activity at early design stages of real-time systems. Based on WCET estimates, engineers make design and implementation decisions to ensure that task executions always complete before their specified deadlines. However, in practice, engineers often cannot provide precise point WCET estimates and prefer to provide plausible WCET ranges. Given a set of real-time tasks with such ranges, we provide an automated technique to determine for what WCET values the system is likely to meet its deadlines, and hence operate safely with a probabilistic guarantee. Our approach combines a search algorithm for generating worst-case scheduling scenarios with polynomial logistic regression for inferring probabilistic safe WCET ranges. We evaluated our approach by applying it to three industrial systems from different domains and several synthetic systems. Our approach efficiently and accurately estimates

Research Activities



Requirements
Quality Assurance

Design-time
Verification

Testing & Analysis

Run-time
Verification

Supporting Safety Analysis of Image-processing DNNs through Clustering-based Approaches

MOHAMMED QUALID ATTAOUI, SnT Centre, University of Luxembourg, Luxembourg, Luxembourg
HAZEM FAHMY, SnT Centre, University of Luxembourg, Luxembourg, Luxembourg
FABRIZIO PASTORE, SnT Centre, University of Luxembourg, Luxembourg, Luxembourg
LIONEL BRIAND, Lero Centre, University of Limerick, Limerick, Ireland and School of EECS, University of Ottawa, Ottawa, Canada

The adoption of **deep neural networks (DNNs)** in safety-critical contexts is often prevented by the lack of effective means to explain their results, especially when there are adversarial. In our previous work, we

Functional Safety of AI-based systems

Security Analysis and Testing

Run-time Monitoring and Trace Checking

Specification Languages

Log Analysis & Anomaly Detection

a
sense
for **IEE**
innovation

Research Activities



Requirements
Quality Assurance

Design-time
Verification

Testing & Analysis

Run-time
Verification

Run-time Monitoring and Trace
Checking

Specification Languages

Log Analysis & Anomaly
Detection

Trace Diagnostics for Signal-based Temporal Properties

Chaima Boufaied, Claudio Menghi, *Member, IEEE*, Domenico Bianculli, *Member, IEEE*,
and Lionel C. Briand, *Fellow, IEEE*

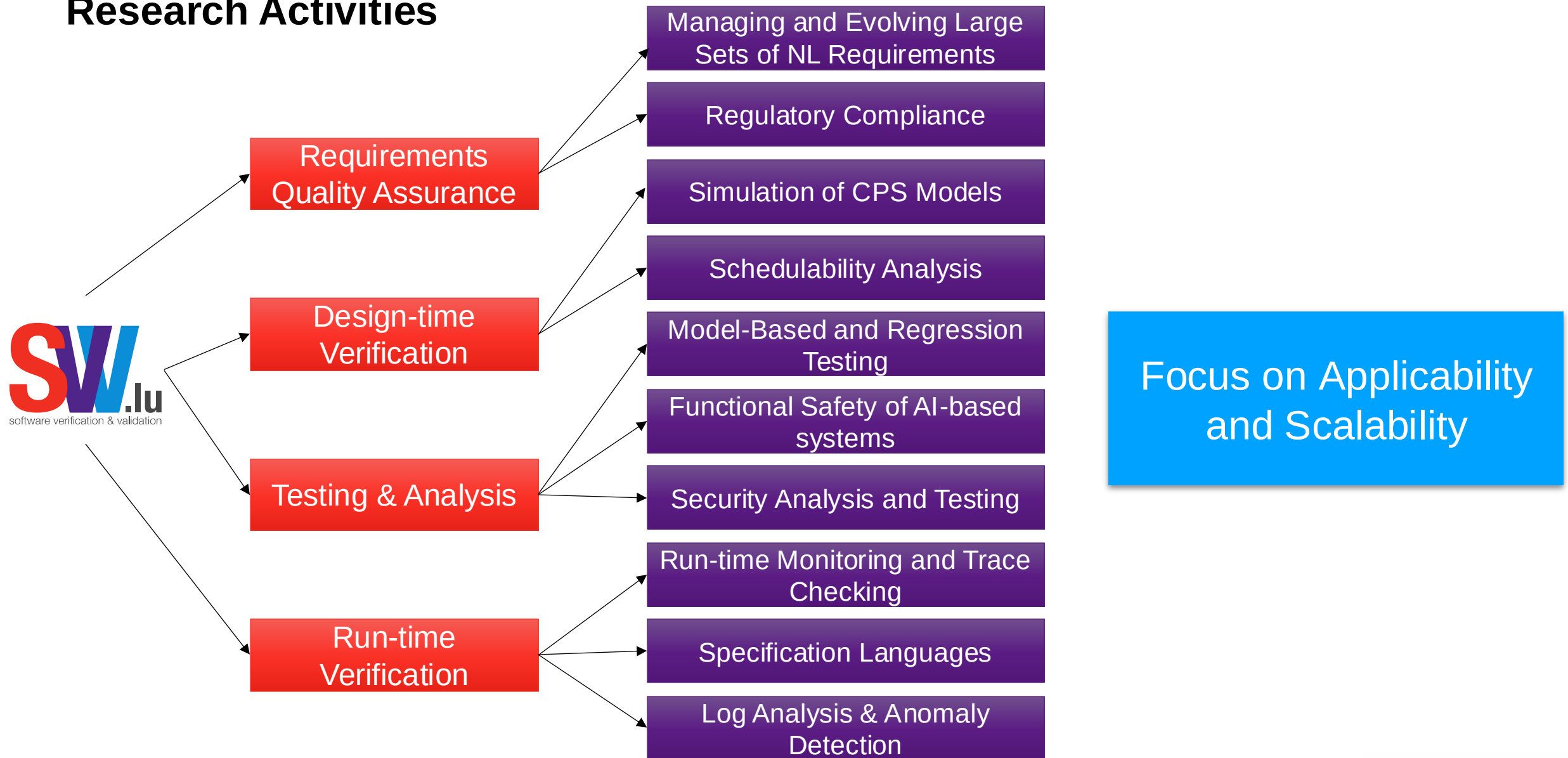
Abstract—

Trace checking is a verification technique widely used in Cyber-physical system (CPS) development, to verify whether execution traces satisfy or violate properties expressing system requirements. Often these properties characterize complex signal behaviors and are defined using domain-specific languages, such as SB-TemPsy-DSL, a pattern-based specification language for signal-based temporal properties. Most of the trace-checking tools only yield a Boolean verdict. However, when a property is violated by a trace, engineers usually inspect the trace to understand the cause of the violation; such manual diagnostic is time-consuming and error-prone. Existing approaches that complement trace-checking tools with diagnostic capabilities either produce low-level explanations that are hardly comprehensible by engineers or do not support complex signal-based temporal properties.

In this paper, we propose *TD-SB-TemPsy*, a trace-diagnostic approach for properties expressed using SB-TemPsy-DSL. Given a property and a trace that violates the property, *TD-SB-TemPsy* determines the root cause of the property violation. *TD-SB-TemPsy* relies on the concepts of *violation cause*, which characterizes one of the behaviors of the system that may lead to a property violation, and *diagnoses*, which are associated with violation causes and provide additional information to help engineers understand the violation cause. As part of *TD-SB-TemPsy*, we propose a language-agnostic methodology to define violation causes and diagnoses. In our context, its application resulted in a catalog of 34 violation causes, each associated with one diagnosis, tailored to properties expressed in SB-TemPsy-DSL.



Research Activities



SVV Mode of Collaboration

Research driven by
industry needs

Realistic evaluations

Research combined with
innovation and
technology transfer

Software Verification and Validation in the Age of GenAI

The Age of GenAI

How did we get there?

Language Models

Increased computational
power

Huge amount of data

The Road to the GenAI Boom



LLM4SE

When LLMs meet Software Engineering

LLM4SE - models

General-purpose models

GPT series

LLaMA

Claude

...

Specialized models

CodeGPT

CodeWhisperer

CodeParrot

Codex

Copilot

...

LLM4SE - training data

Text-based

Programming problems

Requirements documentation

Code Reviews

API documentation

Bug reports

Code-based

Source code

Bug-fix pairs

Test suites

LLM4SE - tasks

Requirement
Engineering

Software Design

Software Development

Software Quality
Assurance

Software Maintenance

Software Management

LLM4SE for Software Quality Assurance

Test generation

Vulnerability detection

Fault localization

Test automation

Static Analysis

Verification

LLM4SE for Software Testing

Unit test generation

Test oracle generation

System test input generation

Test adequacy evaluation

Test minimization

Test flakiness

Towards Generating Executable Metamorphic Relations Using Large Language Models

Seung Yeob Shin¹, Fabrizio Pastore¹,
Domenico Bianculli¹, Alexandra Baicoianu²

¹ SnT/University of Luxembourg, ² Siemens Industry Software

Effective Testing is Hard to Achieve

Software **faults** (functional, security, performance) are **subtle**, they are discovered after testing many inputs

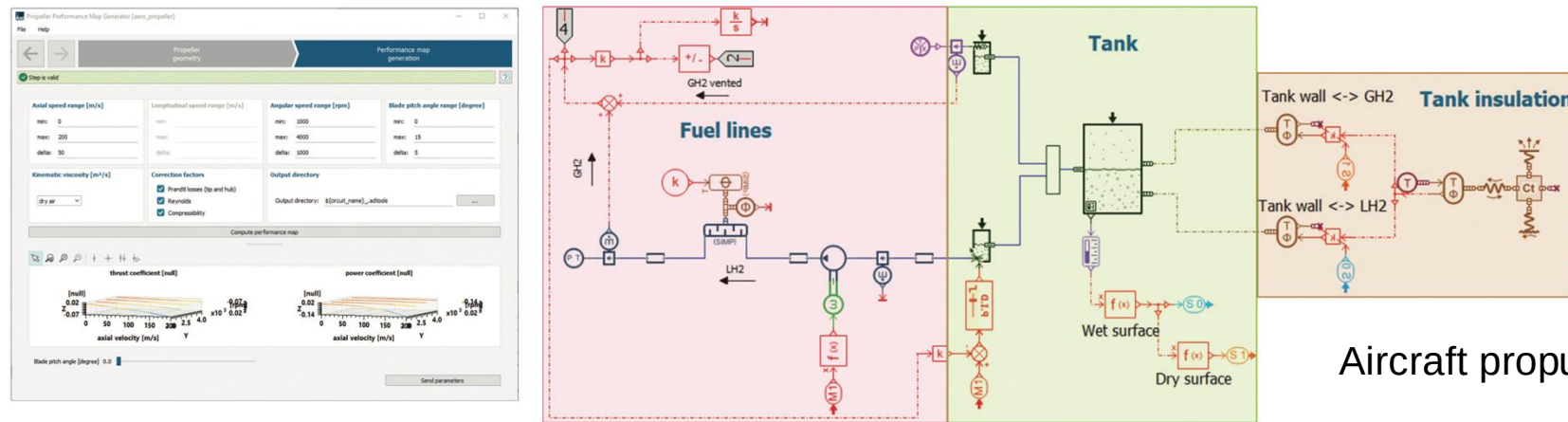
Although we can **automatically** exercise software with many inputs,

automatically determining output correctness is generally **infeasible**

Specifying a **large set of expected results** is impractical

Testing mechatronic system simulation software

- Provide inputs:



Aircraft propulsion system models

- Determine expected outputs (gas emission, energy, heating ...):
expensive
- A nightmare to perform for several inputs

Security Testing of Web Systems: looking for authorization bypass

The image is a collage of Jenkins web interface screenshots, illustrating various parts of the system used for security testing. The screenshots include:

- Jenkins Dashboard:** Shows the main navigation menu with options like New Item, People, Build History, Project Relationships, Check File Fingerprint, and Manage Jenkins.
- People Page:** Displays a list of users, including 'admin' and 'log ou'.
- Build History:** Shows a list of builds with columns for Name, Status, and Date.
- Check File Fingerprint:** A page for checking file fingerprints against the Jenkins database.
- Login Forms:** Multiple instances of the Jenkins login form, showing fields for User and Password, and a 'log in' button.
- Credentials Table:** A table listing stored credentials with columns for Name, Store, Domain, and ID.

The following table is extracted from the 'Credentials' section of the screenshots:

Name	Store	Domain	ID
Jenkins	(global)		e3b49f52-484f-4f93-a478-ee6d0d39eb1

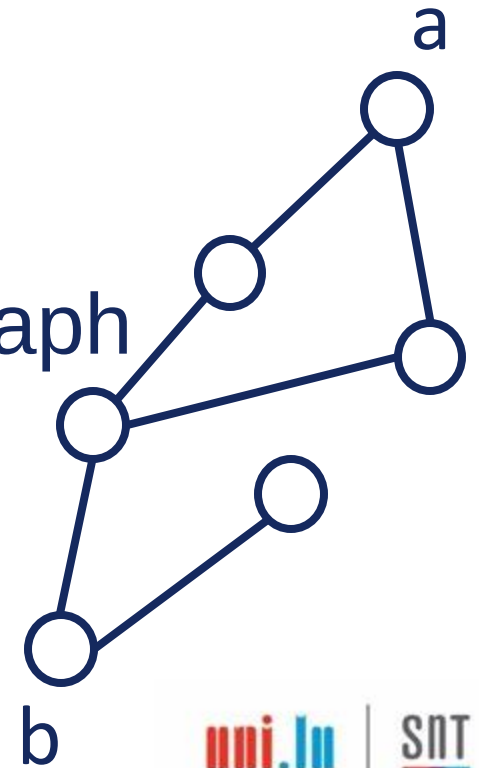
Additional visible text includes 'Create an account if you are not a user', 'log in', and 'SNT' logo.

Metamorphic testing

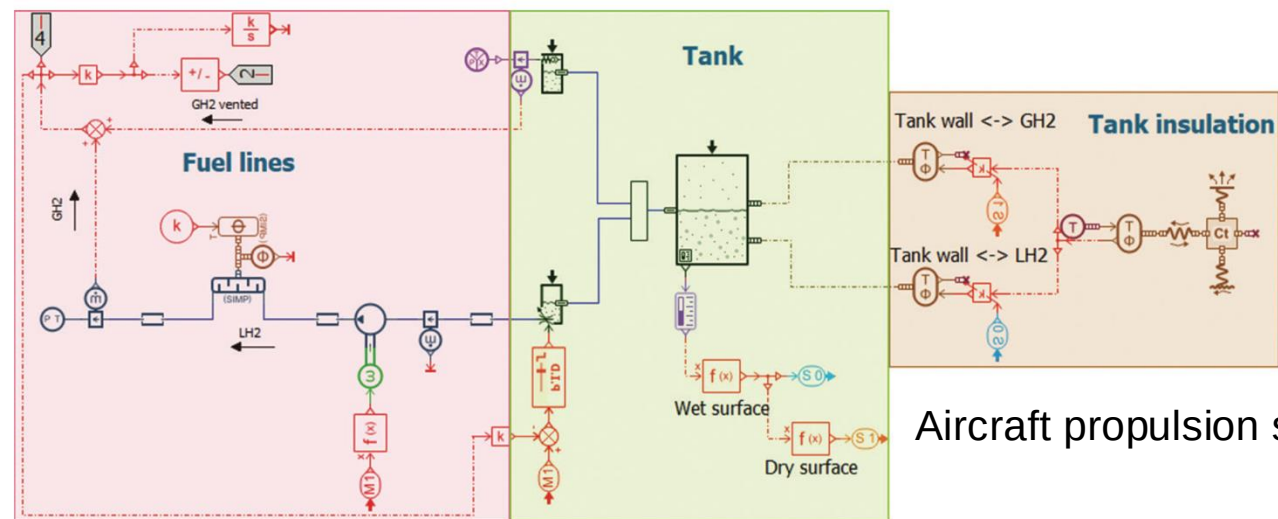
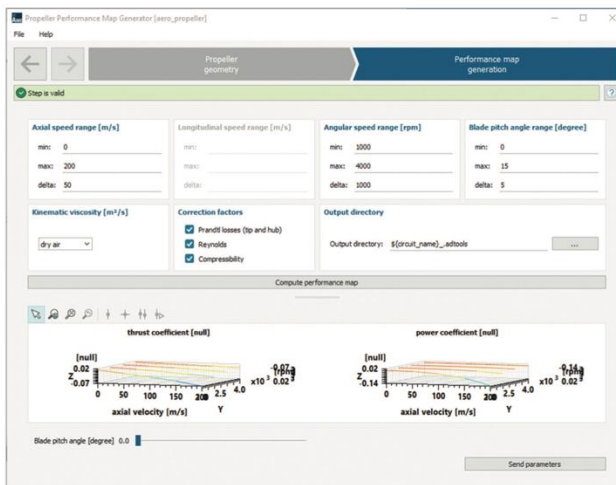
- Rationale: less expensive to verify if relations between **multiple inputs and their outputs** hold, than writing **multiple test assertions** for different inputs
- Example Software Under Test:
 - function for shortest path in an undirected graph
- Example Metamorphic Relation (MR):

G, a, b
↑
Source input

↑
Follow-up input



Metamorphic testing a mechatronic system simulation software



Aircraft propulsion system models

MR: if the propulsion system's fuel type changes from kerosene to hydrogen, then the specific greenhouse gas emissions should decrease

Security Metamorphic Relations

- Source input: a sequence of valid interactions with the system

{**login(Admin)**, RequestURL(**settings_page**)}

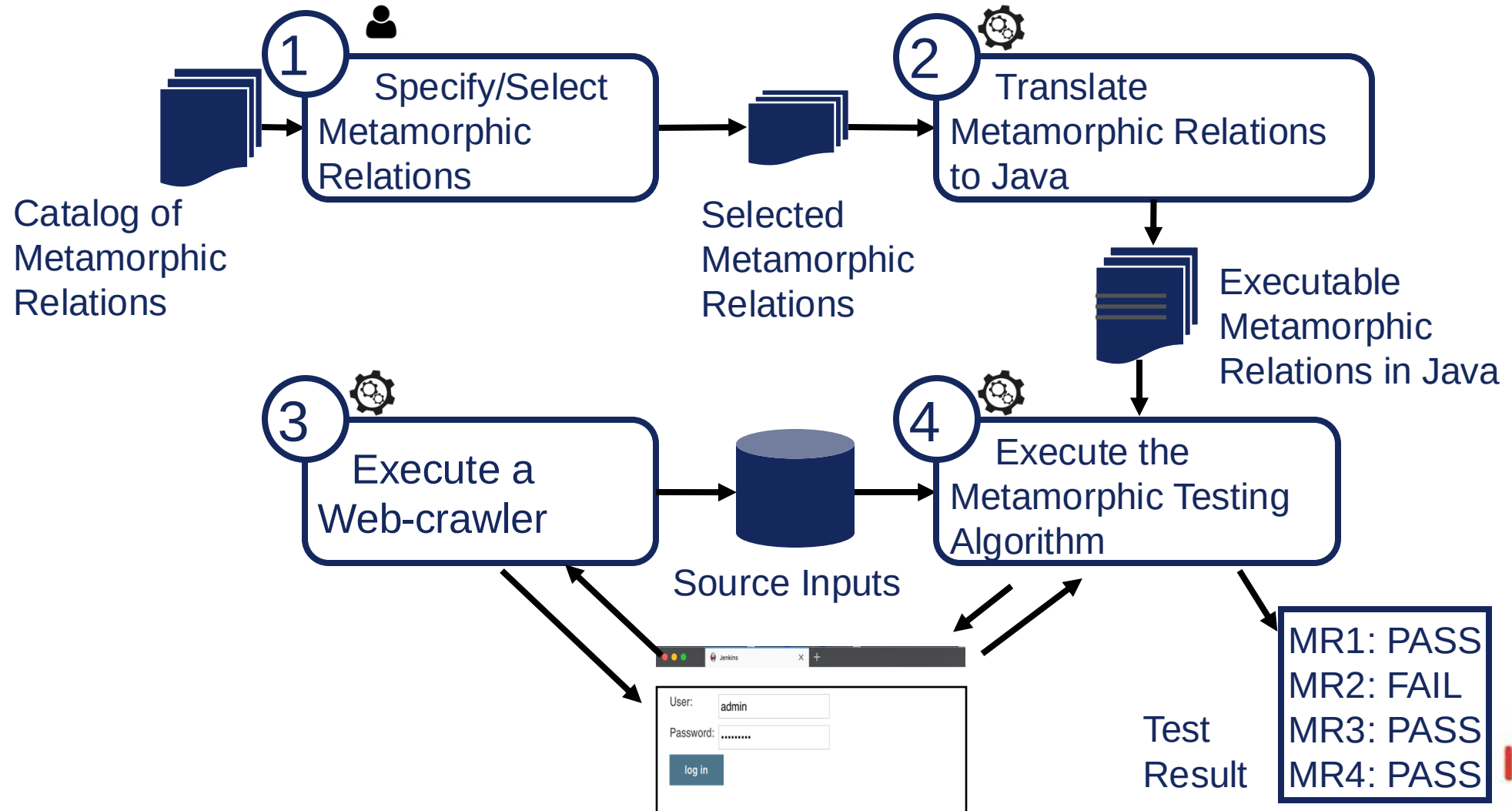
- Follow-up input: generated by altering valid interactions as an attacker would do

{**login(User1)**, RequestURL(**settings_page**)}

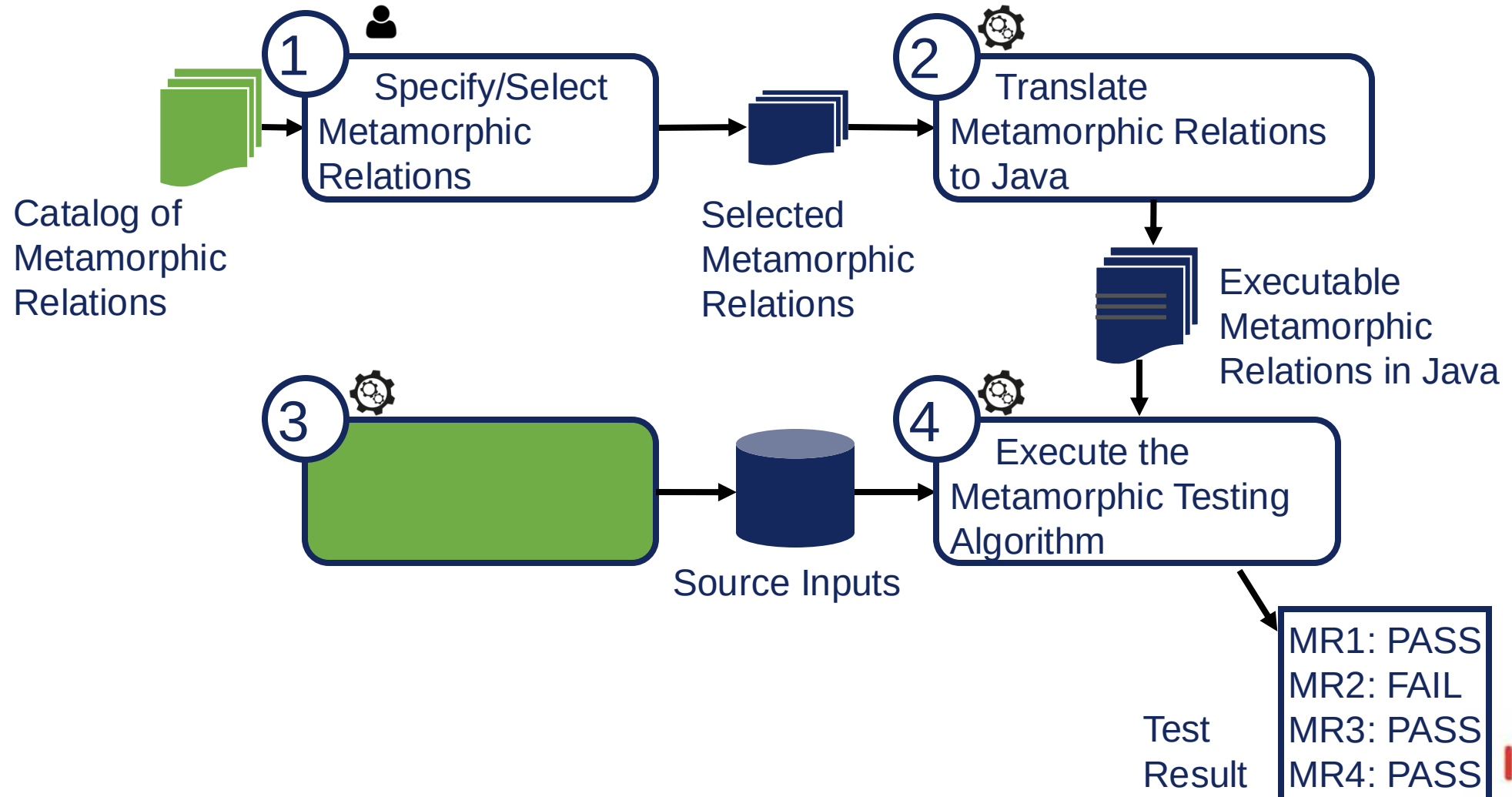
- Relations: capture properties that hold when the system is not vulnerable

if the **user** in the follow-up input **cannot access** the URL from her GUI then the **output** of the source and follow-up inputs should be **different**

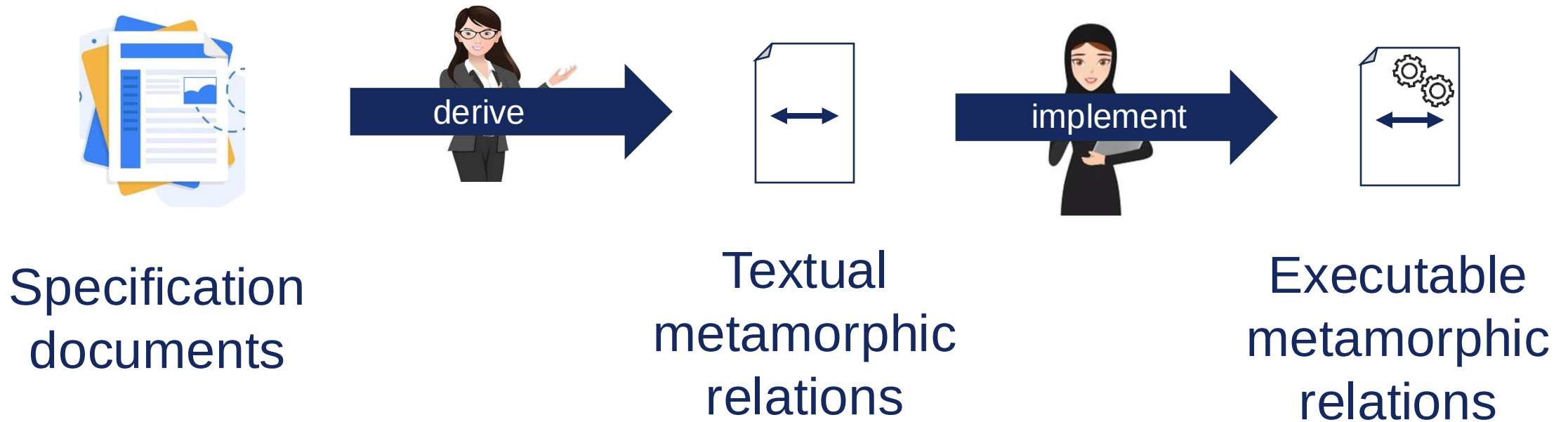
Automated Metamorphic Security Testing Process for Web Systems



Automated Metamorphic Security Testing Process for Other Systems



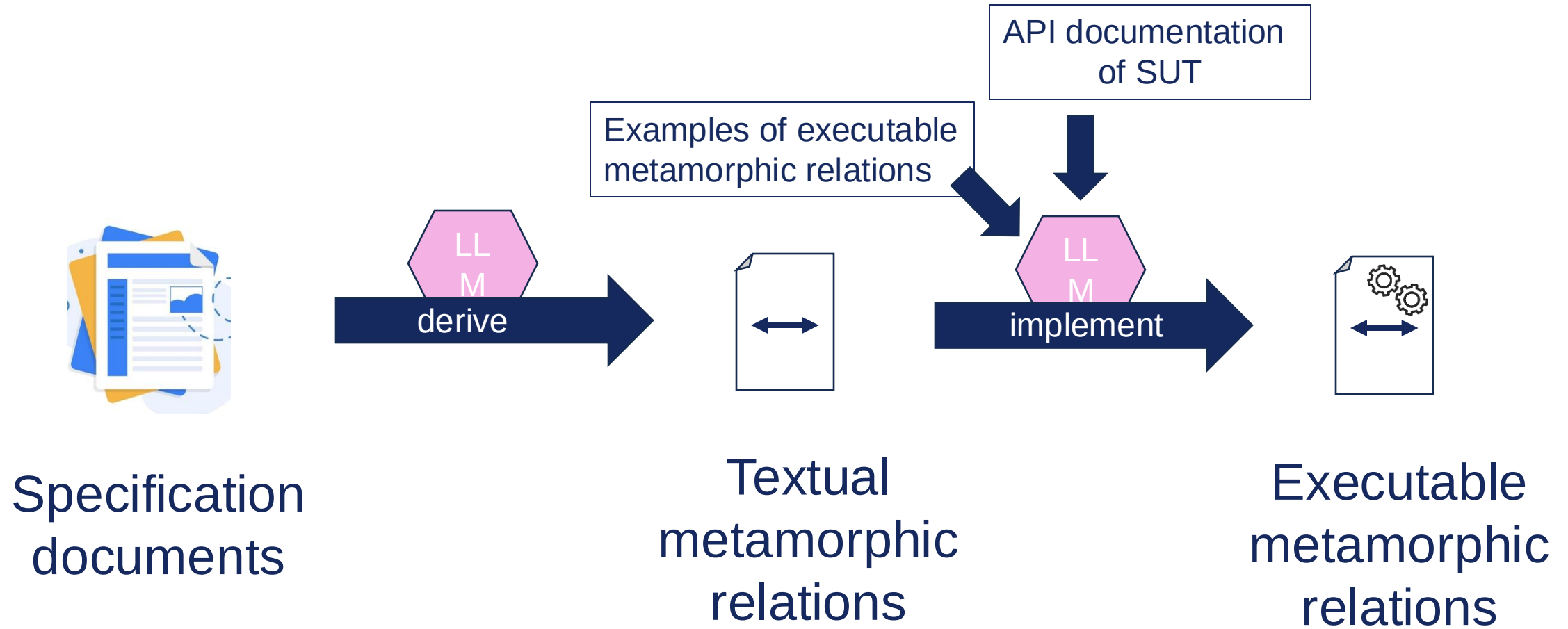
Metamorphic testing limitation: manual effort



Metamorphic testing and LLMs

- Previous work has shown that LLMs can generate MRs for “common test subjects” (e.g., testing traffic control software)
 - Quang-Hung Luu, Huai Liu, and Tsong Yueh Chen. Can ChatGPT advance software testing intelligence? An experience report on metamorphic testing, 2023.
 - Yifan Zhang, Dave Towey, and Matthew Pike. Automated metamorphic relation generation with ChatGPT: An experience report. In COMPSAC, pages 1780–1785, 2023.
- How to deal with **domain-specific software** without specifications or MRs already in the literature/on the Web?

Vision



Approach

Generate textual MRs

Setting the context



I want to derive metamorphic relations for testing purposes from a technical document. Can you assist me?

Certainly! ...

LL
M

Providing software specifications



Here is a technical document. How many pages does it have?

The technical document you provided, ...

LL
M

Generate textual MRs

Identifying relevant statements related to system variables



On pages 1 to 5, can you identify statements that are related to system variables and may be considered by engineers when defining metamorphic relations?

Based on the document, here are key statements related to system variables that engineers may consider when defining metamorphic relations ...

LL
M

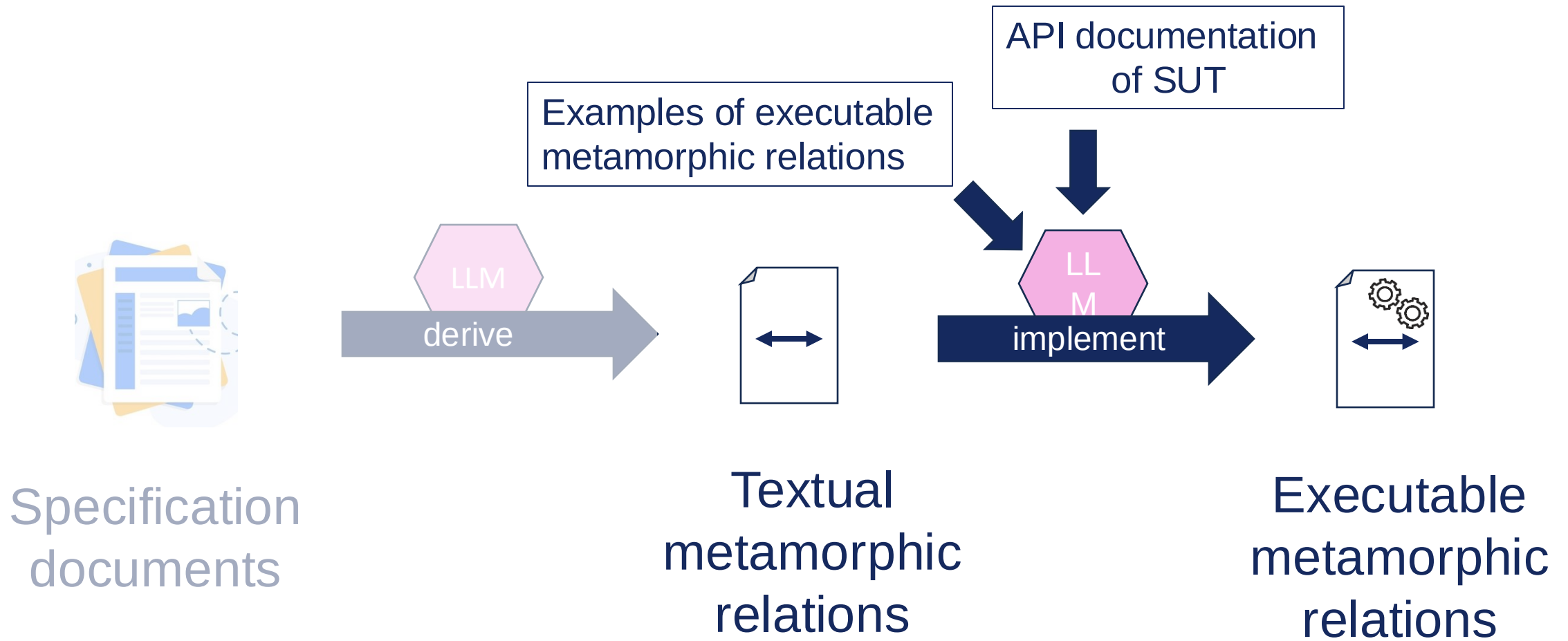
Rewriting them into MRs



Can you rephrase the sentences you found into the form of metamorphic relations?

Certainly! ...

LL
M



Executable MRs

- Implemented with SMRL

- DSL based on Java, from our previous work¹

source inputs

```
MR {{
  Graph graph = Input(1)
  for ( Vertex a : graph.vertices() ){
    for ( Vertex b : graph.vertices() ){
      IMPLIES (
        graph.isUndirected(),
        EQUAL ( graph.shortestPath(a, b ).len(),
                graph.shortestPath( b, a).len() )
      ); //end-IMPLIES
    } //end-for
  } //end-for
}} //end-MR
```

declarative implication

¹Nazanin Bayati Chaleshtari, Fabrizio Pastore, Arda Goknil, and Lionel C. Briand.
Metamorphic testing for web system security. IEEE TSE, 49(6):3430–3471, 2023.

Generate executable MRs

Setting the context



I need your coding capabilities to convert metamorphic relations written in natural language into executable code in a domain-specific language. Can you help me?

Absolutely, ...

LL
M

Learning the syntax
of SMRL



Before we start, here are the notations of SMRL. “Input(int)” is a sequence of Action instances.
“Input(int)” is filled with a randomly selected valid sequence of actions.
“CREATE (Object y , Object x)” defines y as a copy of x. It is..

Certainly! ...

LL
M

Generate executable MRs

Instructing the output format



When I ask you to create an executable MR, please create it by replacing <body> as in the following template: ...

Understood. I'll use the provided template ...



Generate executable MRs

Few-shot learning
of SMRL



Here is the first example of an executable MR written in SMRL. The following MR captures the property “A URL that cannot be reached by a user while navigating the user interface should not be available to that same user even when she directly requests the URL to the server. ...

```
MR  {{
    for (Action action : Input(1).actions()) { //(1)
        IMPLIES( //(2)
            cannotReachThroughGUI (User(), action.url) && //(3)
            CREATE (Input(2), changeCredentials (Input(1), User())) //(4)
        ..

    /* Comments:
    * (1) For loop iterates over all actions of the Input(1)
    * (2) Beginning of implication statement
```

Thank you for providing the example ...

Empirical assessment

Research questions

- RQ1: Can LLM generate textual metamorphic relations from specification documents?
- RQ2: Can LLM generate executable metamorphic relations from textual metamorphic relations?

RQ1 (textual MRs) - Design

- Generate textual MRs with GPT-4 (Jan'24), because it could process documents
- Specifications of different software:

Marine design



DIGITAL INDUSTRIES SOFTWARE

Full-scale simulation for marine design

Increasing the value of CFD for flow around ships and offshore structures

Executive summary
Computational fluid dynamics (CFD) is becoming widely used in marine design. Generally it is run at model scale to compare with testing tank validation data. This paper examines some common reservations about running CFD at full scale and aims to encourage full-scale analysis of designs under realistic operating conditions. In many cases, full-scale simulation is more accurate and reliable than alternatives and leads to greater understanding of design performance.

Professor Milovan Perić

siemens.com/software

SIEMENS

Wind turbine



DIGITAL INDUSTRIES SOFTWARE

Improving wind turbine gear box simulation

ZF Wind Power explores Simcenter 3D Motion transmission builder to enhance drivetrain design

Executive summary
Wind energy production and power output from wind turbines are rapidly increasing. Drivetrains are a critical subsystem for guaranteeing optimal performance for the expected lifetime of these machines. Transmission engineers need to be able to predict the system level to ensure 20 to 25 years of service. More accurate and faster virtual prototyping tools are needed to accomplish this task. This white paper introduces the latest evolution of Siemens Digital Industries Software's drivetrain simulation portfolio. Its capabilities and the significant advances it brings to the field of numerical simulation are shown in the context of a collaboration between Siemens and ZF Wind Power, in which a wind turbine drivetrain system has been simulated using this newly developed method.

siemens.com/simcenter3d

SIEMENS

Aircraft propulsion



DIGITAL INDUSTRIES SOFTWARE

Leveraging preliminary design and simulation to drive next-generation propulsion systems

Using Simcenter Amesim to model a complete hydrogen-powered regional aircraft

Executive summary
This white paper examines the challenges aerospace engineers face that are part of the process of designing sustainable aircraft. With a focus on the early design stage, it investigates using hydrogen-powered jet engines and hydrogen fuel cell technology in driving next-generation propulsion systems and their implications for subsystems, culminating in the need to reimagine aircraft configurations. Using Siemens Digital Industries Software's Simcenter® software helps you support digital twin technology to help aerospace engineering organizations optimize aircraft performance using virtual and physical testing of the fluid, thermal, mechanical and other system domains impacted by green aviation. Simcenter is part of the Siemens Xcelerator business platform of software, hardware and services.

siemens.com/simcenter-amesim

SIEMENS

Aero-acoustics

High-order unstructured methods for computational aero-acoustics

Hadrien Bériot¹, Gwénaél Gabard², Karim Hamiche¹, Michael Williamschen²

¹Siemens PLM, Interleuvenlaan 68, Researchpark Z1, Leuven, Belgium

²Institute of Sound and Vibration Research, University of Southampton, Southampton, SO17 1BJ, United Kingdom

November 17, 2015

Contents

1 Introduction	3
2 Modeling flow acoustics	4
2.1 The physical models	4
2.2 Linearised Euler Equations	5
2.3 Linearised potential theory	6
3 The numerical methods	7
3.1 Frequency vs. time domain	7
3.2 Structured vs. Unstructured	7
3.3 Computational schemes	8
3.4 Modeling unbounded domains	9
3.5 Mesh generation algorithms	10
4 A frequency-domain scheme (FEM)	10
4.1 The weak form formulation	11
4.2 Potential theory formulation	11
4.3 LEE formulation	12
4.4 Discrete model	13
4.5 Shape functions	14
4.6 Sparse system solvers	15
4.7 Numerical accuracy	17
4.8 Application in aeroacoustics	21
4.8.1 Air intake	21
4.8.2 Exhaust radiation	24

*The authors acknowledge the European Commission (EC) for the support of the FLOWAIRS (www.flowairs.eu) and the CRANE projects (www.crane-td.eu).

VKI

- 1 -

- Questionnaire-based survey with engineers at Siemens Industry Software

RQ1 (textual MRs) - Design

- 3 engineers
- 64 metamorphic relations

MR Questionnaire

Please indicate your level of agreement with the statements below, corresponding to metamorphic relations (MRs) derived from a technical document. Your feedback is valuable and will help us better understand your perspective.

Document: Full-scale simulation for marine design

MR 1	If the quality and resolution of the numerical grid or the turbulence models are varied, the solution's accuracy should change correspondingly, while maintaining consistency with physical principles.
Statement 1	The MR is easy to understand. ☐ Strongly Agree ☐ Agree ☐ Neutral ☐ Disagree ☐ Strongly Disagree
Statement 2	The MR captures a property that needs to be considered when testing the system. ☐ Strongly Agree ☐ Agree ☐ Neutral ☐ Disagree ☐ Strongly Disagree
Statement 3	The MR helps identify the expected outputs for given inputs. ☐ Strongly Agree ☐ Agree ☐ Neutral ☐ Disagree ☐ Strongly Disagree

RQ1 (textual MRs) - Results

	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
The MR is easy to understand	3	46 (77%)	12	3	0

Engineers can understand the presented MRs

RQ1 (textual MRs) - Results

	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
The MR is easy to understand	3	46	12	3	0
The MR captures a property that needs to be considered for testing	0	41 (64%)	10	12	1

The MRs captures properties that need to be considered when testing the SUT

RQ1 (textual MRs) - Results

	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
The MR is easy to understand	3	46	12	3	0
The MR captures a property that needs to be considered for testing	0	41	10	12	1
The MR helps identify the expected outputs for given inputs	0	18	19	24	3

MR: “Different types of fuel cells should yield **varying** levels of system efficiency and output”

- Mixed answers
- MRs are too generic to identify what to assess
- May depend on our specification documents

RQ1: Answer

	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
Statement 1	3	46	12	3	0
Statement 2	0	41	10	12	1
Statement 3	0	18	19	24	3

LLMs can generate correct textual MRs, but these may need to be improved with additional details

RQ2 (executable MRs) Design

- We considered previous work with textual MRs and corresponding executable MRs
- We generated executable MRs from 10 textual MRs in previous work
- We manually compared each generated MR with the MR manually implemented
 - labelled each statement

Labels:

- WI: The statement does not correctly implement what suggested in the explanation
- WS: The statement does not make sense, and what is suggested in the explanation makes no sense for this
- IE: The statement invokes an API function invented by GPT even if an adequate one exists
- INE: The statement invokes an API function invented by GPT because an adequate one does not exist
- ITE: The statement invokes an API function invented by GPT but delegates too much logic to it
- ES: The statement invokes an existing and appropriate API function but swaps parameters
- ENO: The statement invokes an existing and appropriate API function but not in an object-oriented manner
- WAU: The statement misuses a valid API
- C: Correct statement
- AI: Statement that does not perform exactly what done in the original MR but might be a valid alternative implementation
- CLC: Correct language construct (e.g., IMPLIES, NOT, EQUAL, MR)
- WLC: Wrong usage of a language construct (WLC)
- MISS: One instruction, required to implement what reported in the explanation, is missing

Alzette, Luxembourg (e-mail: fabrizio.pastore@uni.lu).
Arda Goknil is with the Sustainable Communication Technologies, SINTEF Digital, 0373 Trondheim, Oslo, Norway (e-mail: Arda.Goknil@sintef.no).
Lionel C. Briand is with the SoT Centre, University of Luxembourg, 4365 Esch-sur-Alzette, Luxembourg (e-mail: lionel.briand@uni.lu).
Digital Object Identifier 10.1109/TSE.2023.3256322

0098-5589 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Luxembourg. Downloaded on February 28, 2024 at 16:51:46 UTC from IEEE Xplore. Restrictions apply.

```
//end-TOI
}
} //end-MR
} //end-package
```

RQ2 (executable MRs) Results

Generated executable MRs: total 138 statements (for each, min 10, avg 13.8, max 20)

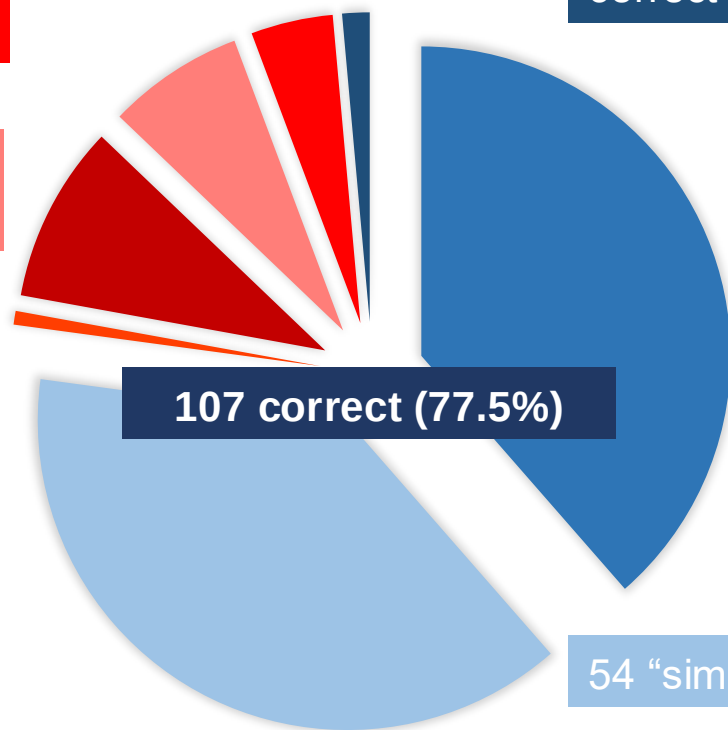
6 statements incorrectly implement what is suggested in the corresponding explanations

10 statements incorrectly use SMRL constructs

13 statements incorrectly use APIs

2 statements miss implementing what is described in the corresponding explanations

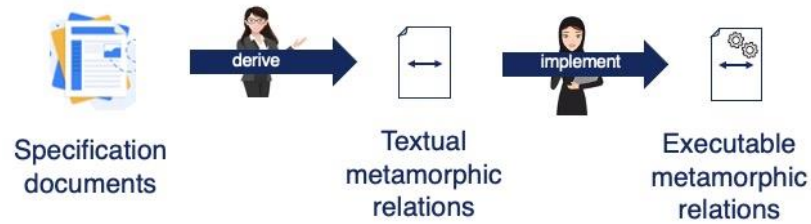
1 does not match the reference executable MR but is correct



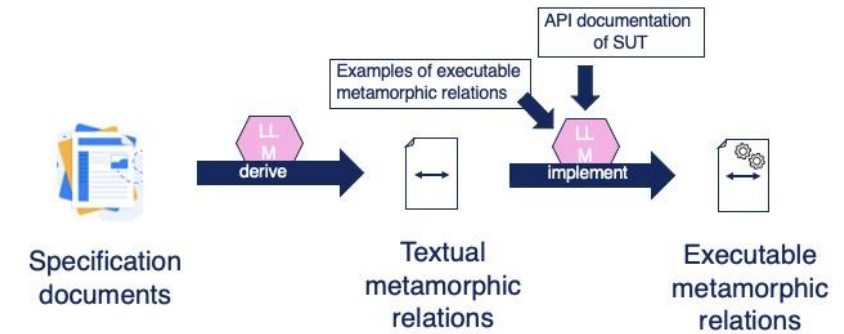
52 “complex” statements with function calls and match the reference executable MR

54 “simple” statements with SMRL constructs

Metamorphic testing limitation: manual effort



Vision



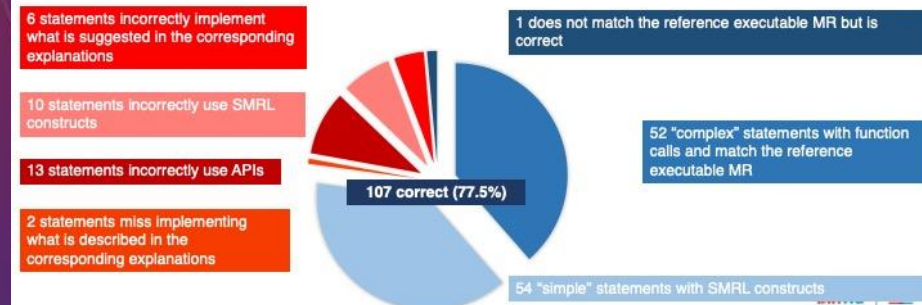
RQ1: Answer

	Strongly agree	Agree	Neutral	Disagree	Strongly disagree
Statement 1	3	46	12	3	0
Statement 2	0	41	10	12	1
Statement 3	0	18	19	24	3

LLMs can generate correct textual MRs, but these may need to be improved with additional details

RQ2 (executable MRs) Results

Generated executable MRs: total 138 statements (for each, min 10, avg 13.8, max 20)





Software Verification and Validation (SVV)

<https://www.uni.lu/snt-en/research-groups/SVV/>

Contact:



Domenico Bianculli

Head of SVV

domenico.bianculli@uni.lu

